

PART-A

1. Relate the terms Database and Database management Systems.

A "database" is essentially a collection of organized data stored electronically, while a "Database Management System (DBMS)" is the software that allows users to create, access, manipulate, and manage that data within the database.

2. List the advantages of DBMS.

Controlling of Redundancy: Data redundancy refers to the duplication of data (i.e. storing same data multiple times). In a database system, by having a centralized database and centralized control of data by the DBA the unnecessary duplication of data is avoided. It also eliminates the extra time for processing the large volume of data. It results in saving the storage space.

Improved Data Sharing : DBMS allows a user to share the data in any number of application programs.

Data Integrity : Integrity means that the data in the database is accurate. Centralized control of the data helps in permitting the administrator to define integrity constraints to the data in the database. For example: in customer database we can enforce an integrity that it must accept the customer only from Noida and Meerut city.

3. Define instances and Schemas of database.

INSTANCE: Databases change over time as information is inserted and deleted. The collection of information stored in the database at a particular moment is called an instance of the database.

SCHEMA: The overall design of the database is called the database schema.

4. Give an example for total participation and partial participation.

Example:

Suppose an entity set Student related to an entity set Course through Enrolled relationship set.

The participation of entity set course in enrolled relationship set is **partial** because a course may or may not have students enrolled in. It is possible that only some of the course entities are related to the student entity set through the enrolled relationship set.

The participation of entity set student in enrolled relationship set is **total** because every student is expected to relate at least one course through the enrolled relationship set.



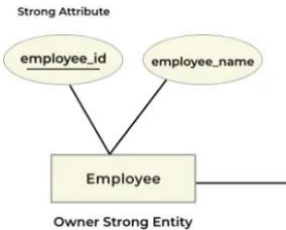
Participation in Enrolled relationship set: **Partial** Course
Total Student

5. How to represent the strong Entity set and Weak entity set in ER-Model?

Strong entity

- Strong Entity is independent of any other entity in the schema

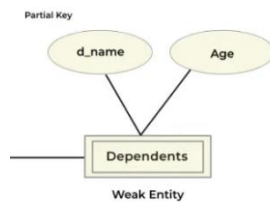
Example – A student entity can exist without needing any other entity in the schema or a course entity can exist without needing any other entity in the schema.



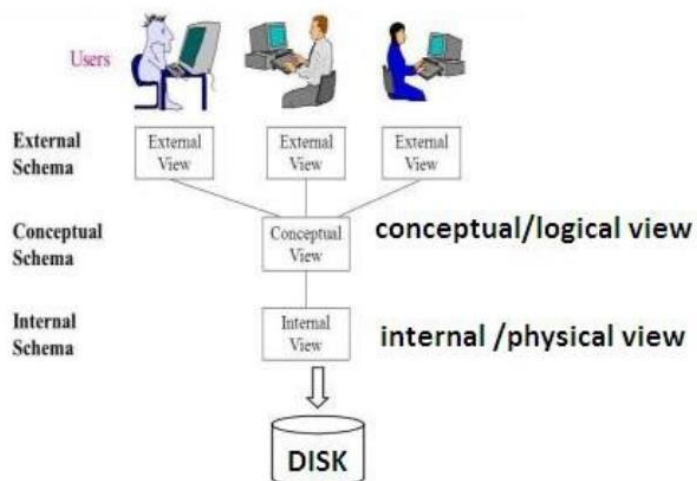
Weak entity

- A weak entity is an entity set that does not have sufficient attributes for Unique Identification of its records.

Example 1 – A loan entity can not be created for a customer if the customer doesn't exist



6. Draw the diagram for levels of abstraction and explain.



- **Physical level (or Internal View / Schema):** The lowest level of abstraction describes how the data are actually stored.

• **Logical level (or Conceptual View / Schema):** The next-higher level of abstraction describes what data are stored in the database, and what relationships exist among those data.

• **View level (or External View / Schema):** The highest level of abstraction describes only part of the database which are requested by the user, but not the entire database.

7. List the integrity constraints in relational model.

Integrity constraints are a set of rules that are used to maintain the quality of information.

There are many types of integrity constraints.

- 1. Entity Integrity Constraint/Key constraints
- 2. Referential Integrity constraints/ Foreign key
- 3. Domain constraints
- 4. General constraints

8. Give examples of selection and projection operations in relational algebra.

Selection (σ): Selection operator is used to select tuples from a relation based on some condition. A new relation is created by selecting only those rows requested by the user satisfying a specified condition.

Syntax: σ (Cond)(Relation Name)

E.g: σ (AGE>18)(STUDENT)

Projection (π): The projection eliminates all attributes of the input relation but those mentioned in the projection list. That means all other unnecessary fields are projected out.

Syntax:

Π (Column 1,Column 2....Column n)(Relation Name)

E.g: Extract ROLL_NO and NAME from STUDENT relation

Π (ROLL_NO,NAME)(STUDENT)

9. What is Querying relational data? Justify with one example.

Querying relational data refers to the process of retrieving and manipulating data from a relational database using a query language, most commonly **SQL (Structured Query Language)**. A relational database stores data in tables (relations), and querying involves specifying what data you want to retrieve or modify, along with conditions and criteria.

Justification:

Querying relational data is fundamental because it allows users to:

- Retrieve specific data from large datasets.
- Perform complex calculations or operations.
- Filter, group, and sort data as needed.
- Maintain data integrity by working within the structure of relationships and constraints.

This process is essential in various applications like business analytics, reporting, and data management.

10. Illustrate division Operation in relational algebra.

In **relational algebra**, the **division operation** is used when we need to find records from one relation that are related to **all records** in another relation. It is typically used in queries where we want to find entities that satisfy certain conditions with **every** element of another set.

Syntax of Division:

The division operation is denoted by the symbol $\div$ and is typically used in situations involving two relations, say **R** and **S**.

Relation **R** / Relation **S**

- The result will be the set of tuples from **R** that are related to all tuples in **S**.

11 Write short notes on altering tables and views.

Altering Tables

In a relational database, **altering a table** means modifying its structure after it has been created. This can involve several operations such as adding, deleting, or modifying columns, or even changing constraints. These operations are typically done using the **ALTER TABLE** statement in SQL.

Altering Views

A **view** is a virtual table that is based on the result of a **SELECT** query. **Altering a view** involves modifying the definition of the view without changing the actual underlying tables. The most common operation is to **redefine** or **update** the view's **SELECT** statement.

12 Explain Domain Relational Calculus.

Domain Relational Calculus (DRC) is a non-procedural query language used to specify queries on relational databases. Unlike relational algebra, which focuses on operations and the sequence in which they should be performed, **domain relational calculus** describes the *conditions* that data must satisfy in order to be retrieved, without specifying *how* to retrieve it.

13 Give examples for UNION, INTERSECT and EXCEPT Clauses.

Example:

Consider two tables: **Employees** and **Contractors**.

Employees Table:

EmployeeID	Name	Department
1	Alice	HR
2	Bob	IT
3	Charlie	Finance

Contractors Table:

ContractorID	Name	Department
101	David	HR
102	Eva	IT

UNION

Example: We want to combine the list of **Names** from both tables.

Sql: SELECT Name FROM Employees UNION SELECT Name FROM Contractors;

Result:

Name
Alice
Bob
Charlie
David
Eva

INTERSECT :

Example: Using the same **Employees** and **Contractors** tables, suppose we want to find the **Names** that appear in both tables (i.e., people who are both employees and contractors).

Sql: SELECT Name FROM Employees INTERSECT SELECT Name FROM Contractors;

Result:

Name
David

EXCEPT

Example: Again, using the **Employees** and **Contractors** tables, suppose we want to find the **Names** of people who are **only employees** and not contractors.

Sql: SELECT Name FROM Employees EXCEPT SELECT Name FROM Contractors;

Result:

Name
Alice
Bob
Charlie

14 Define Aggregate Operators.

Aggregate operators are functions that perform a calculation on a set of values and return a single result.

COUNT():Returns the number of rows in a table (or group of rows in a query)

SUM():Returns the sum of the values in a numeric column

AVG():Returns the average value of a numeric column

MIN():Returns the smallest (minimum) value in a column

MAX():Returns the largest (maximum) value in a column.

15 Demonstrate how to add a NOT NULL Column to a Table.

To **add a NOT NULL column** to an existing table in SQL, we use the ALTER TABLE statement. When adding a new column to a table, if the column is defined as **NOT NULL**, it means that every row in the table must have a value for that column. If the column is to be added to a table that already contains data, we need to make sure that either:

1. The column has a default value, or
2. We specify a value for the existing rows before setting the column to **NOT NULL**.

Syntax: ALTER TABLE table_name ADD column_name data_type NOT NULL;

PART-B

1.Compare and Contrast File Systems with Database Systems.(4)

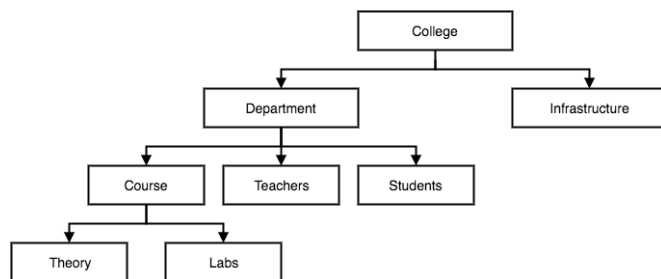
Feature	File System	DBMS
Data Storage	Data stored in independent files (e.g., .txt, .jpg).	Data is stored in tables (rows and columns).
Example	A folder with files like "customer1.txt", "customer2.txt".	A table named "Customers" with columns: ID, Name, Account Balance.
Data Relationships	No inherent relationships between files.	Supports relationships (e.g., foreign keys).
Example	No connection between customer1.txt and customer2.txt. 	"Customer" table linked to "Orders" table using CustomerID as foreign key.
Data Integrity	Minimal support for data validation or consistency.	Strong data integrity with constraints like primary keys, foreign keys, and unique constraints.
Example	No enforcement of unique customer IDs in customer.txt.	Ensures no two customers in "Customers" table have the same CustomerID.
Security	File-level security with permissions (read/write).	Granular security (role-based access, encryption).
Example	You can set "read-only" permission on a file. 	DBMS can restrict access to certain tables based on user roles (e.g., Admin, User).

2. Discuss about different types of Data Models.(4)

- A Data Model defines the structure, connections, storage, access, and modifications of data in a DBMS.
- It visually represents how data is connected and organized using symbols and language, ensuring clarity and effective communication within an organization.
- There are 5 types of Data models.

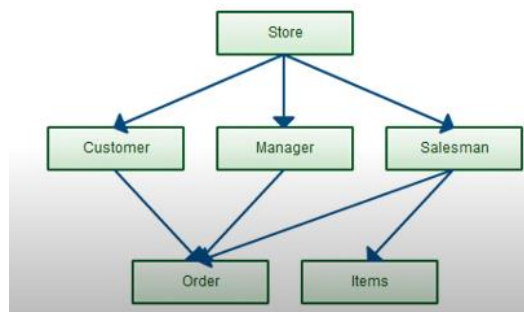
i. Hierarchical Model:

- The Hierarchical Model is a data model in which data is organized in a tree-like structure with a parent-child relationship.
- Each parent can have multiple children, but each child has only one parent. i.e. one to many relationship.
- Data is connected through links or pointers, making traversal efficient for hierarchical relationships.



Network model:

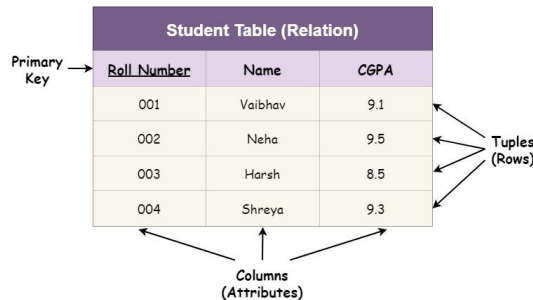
- The Network Model is a data model where data is organized using a graph-like structure with many-to-many relationships.
- A child can have multiple parents, allowing for more flexible connections between data.
- Data is connected using pointers (links), making traversal efficient for complex relationships.
- It is invented by Charles Bachmann.



ii. RelationalModel:

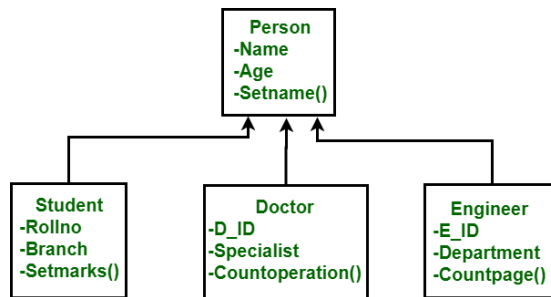
- It is invented by Edgarr F .codd in 1969.
- The Relational Model organizes data into tables (relations) consisting of rows (tuples) and columns (attributes).
- Each table has a unique key to identify records, and relationships between tables are established using foreign keys.
- This model ensures data integrity, flexibility, and easy retrieval using Structured Query Language (SQL).

Relational Model in DBMS



iii. Object-oriented model:

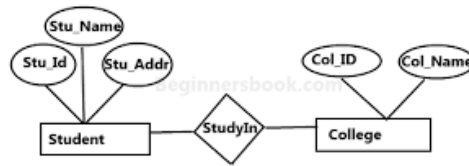
- The Object-Oriented Model stores data as objects, just like in object-oriented programming.
- Each object has attributes (data) and methods (actions) that define its behaviour.
- This model allows data to be organized in a way that reflects real-world entities and their relationships.



iv. Entity-Relationship model:

- The Entity-Relationship (ER) Model is a way to visually represent the structure of a database of a real world problem.
- It uses entities (objects or things in the real world) and relationships (connections between entities).
- Entities are represented as rectangles and can be things like Customers, Orders, or Products.
- Attributes are the properties of entities (e.g., Customer Name, Order Date).

- Relationships are shown as diamonds, indicating how entities are connected (e.g., a Customer places an Order).



Sample E-R Diagram

3. Write about various database system applications in detail.(4)

APPLICATIONS:

Sales: For customer, product, and purchase information.

◦ **Accounting:** For payments, receipts, account balances, assets and other accounting information.

◦ **Human resources:** For information about employees, salaries, payroll taxes, and benefits, and for generation of paychecks.

◦ **Manufacturing:** For management of the supply chain and for tracking production of items in factories, inventories of items in warehouses and stores, and orders for items.

Online retailers: For sales data noted above plus online order tracking, generation of recommendation lists, and maintenance of online product evaluations.

◦ **Banking:** For customer information, accounts, loans, and banking transactions.

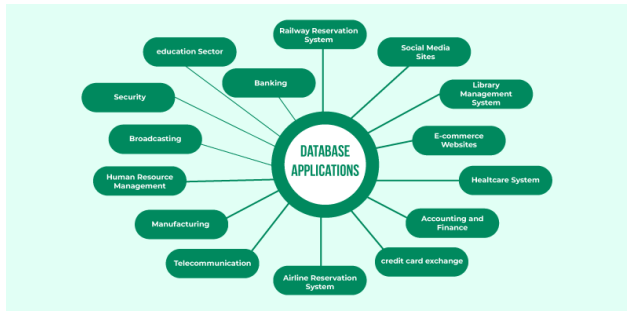
◦ **Credit card transactions:** For purchases on credit cards and generation of monthly statements.

◦ **Finance:** For storing information about holdings, sales, and purchases of financial instruments such as stocks and bonds; also for storing real-time market data to enable online trading by customers and automated trading by the firm.

• **Universities:** For student information, course registrations, and grades (in addition to standard enterprise information such as human resources and accounting).

• **Airlines:** For reservations and schedule information. Airlines were among the first to use databases in a geographically distributed manner.

• **Telecommunication:** For keeping records of calls made, generating monthly bills, maintaining balances on prepaid calling cards, and storing information about the communication networks.



4. Draw and explain Data Abstraction and Data Independence in detail.(4)

DATA ABSTRACTION:

- Data abstraction is the process of hiding unwanted and irrelevant details from the end user.
- It helps to store information in such a way that the end user can access data which is necessary, the user will not be able to see what data is stored or how it is stored in a database.
- Data abstraction helps to keep data secure from unauthorized access and it hides all the implementation details.

Levels of Abstraction:

- There are three levels of data abstraction in DBMS that are mentioned below:

a) Physical or Internal Level:

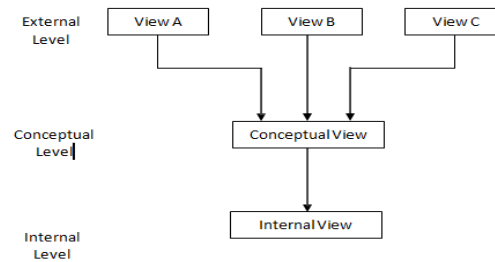
- It is the lowest level of data abstraction which defines how data is stored in database.
- It defines data structures used to store data and methods to access data in database.
- It is very complex to understand and hence kept hidden from user.
- The internal view is described by internal schema.

b) Logical or Conceptual Level:

- It is intermediate level present next to physical level.
- It defines what data is present in database and their relationships between them .
- It is less complex as compared to physical level.
- Programmers generally work at this level and depending on data, structure of tables, relationships and their constraints is decided at this level.
- Conceptual view is defined by conceptual schema.

c) View or External Level:

- It is the highest level in abstraction.
- There are different levels of views and each view defines only a part of whole data required to user.
- This level defines many views of same database for simplification of view to user.
- This is the highest level and easiest to understand for user.
- External view is describes by external schema.



DATA INDEPENDENCE:

- Data Independence is the ability to change the schema at one level of a database system without affecting the schema at the next higher level.
- It is important because it allows changes in the data structure without disrupting application programs or the way users interact with the data.

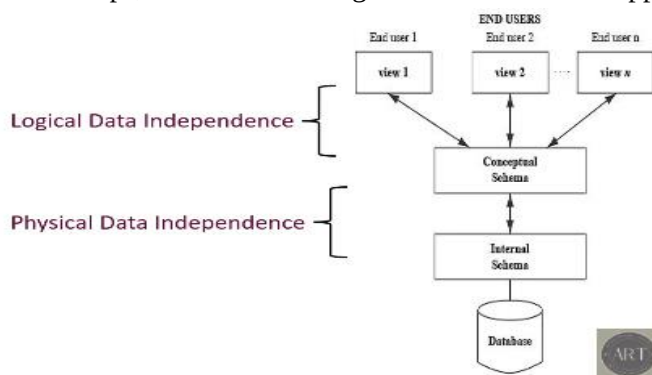
Types of Data Independence:

❖ Physical Data Independence:

- The ability to change the physical storage of data (how data is stored) without affecting the logical schema (what data is stored and its relationships).
- It allows changes in the hardware or file organization without impacting application programs or user views.

❖ Logical Data Independence:

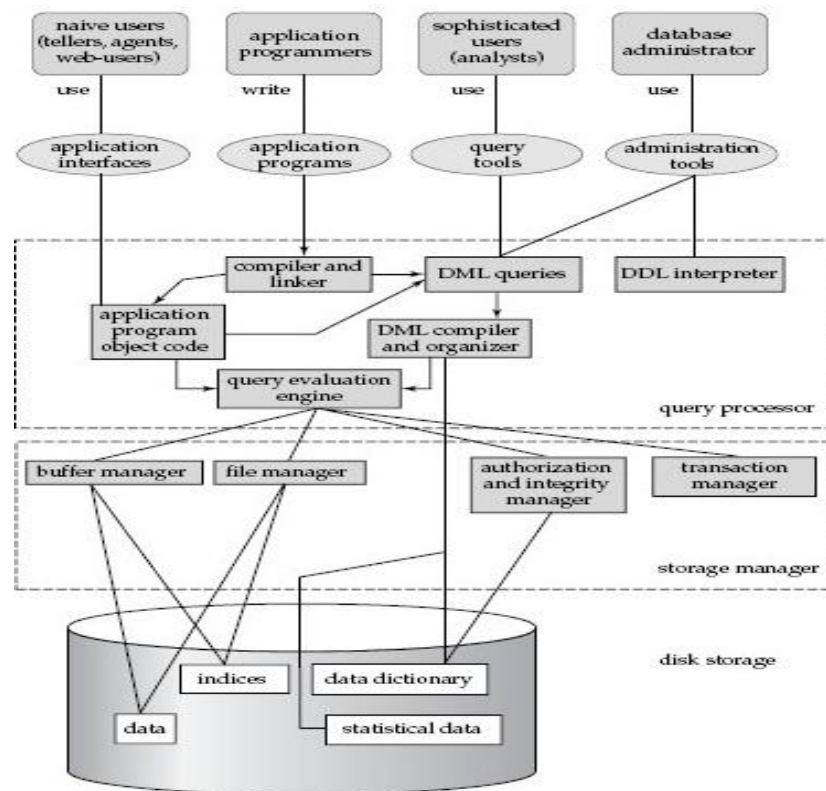
- The ability to change the logical schema (the structure of data and relationships) without affecting the external schema or application programs.
- It provides flexibility in modifying the database structure, such as adding new fields or changing relationships, without disturbing the user interface or applications.



5. Describe the Structure of DBMS.(8)

ANS:

- A Database Management System (DBMS) is software that allows users to define, store, maintain, and manage data in a structured and efficient manner.
- A database system is partitioned into modules that deal with each of the responsibilities of the overall system.
- The functional components of a database system can be broadly divided into the storage manager, the query processor components and disk storage.



❖ Application Programmers

- Develop application programs for interacting with databases.
- Use tools like Rapid Application Development (RAD) to create forms and reports without coding.

❖ Sophisticated Users

- Directly interact with the database using query languages like SQL.
- Submit queries to the query processor, which converts them into instructions for the storage manager.

❖ **Specialized Users**

- Create advanced database applications that go beyond traditional data processing.
- Work on CAD systems, expert systems, multimedia databases, and environment modelling systems.

❖ **Naïve Users**

- Use predefined application programs to interact with the database.
- Example: A bank teller using a transfer program to move money between accounts.

❖ **Database Administrator (DBA)**

- Manages the entire database system and its resources.
 - Schema Definition & Modification:** The DBA designs the database schema and updates it as per evolving business needs.
 - Storage Structure & Access Control:** Manages data storage methods and controls access to ensure efficient and secure data retrieval
 - User Authorization & Security:** Assigns user permissions to regulate database access and protect sensitive information.
 - Performance Monitoring & Optimization:** Continuously analyses and improves database performance by tuning queries and optimizing storage.
 - Ensuring Data Integrity:** Implements constraints and rules to maintain accuracy, consistency, and reliability of the database.

Query Processor:-

- The query processor will accept query from user and solves it by accessing the database.
- The Query Processor simplifies and facilitates data access in a database system.
- It ensures quick processing of updates and queries by translating high-level commands into efficient low-level operations.
- This improves database performance and efficiency.

Query Processor Components:

- **DDL interpreter:** It interprets DDL statements and records the definitions in the data dictionary.
- **DML compiler:** It translates DML statements in a query language into an evaluation plan consisting of low-level instructions that the query evaluation engine understands.
- **Query evaluation engine:** It executes low-level instructions generated by the DML compiler.

Storage manager:

- A storage manager is a program module which acts like interface between the data stored in a database and the application programs and queries submitted to the system.
- Thus, the storage manager is responsible for storing, retrieving and updating data in the database.

Storage manager components :

- **Authorization and integrity manager:** Checks for integrity constraints and authority of users to access data.
- **Transaction manager:** Ensures that the database remains in a consistent state although there are system failures.
- **File manager:** Manages the allocation of space on disk storage and the data structures used to represent information stored on disk.
- **Buffer manager:** It is responsible for retrieving data from disk storage into main memory. It enables the database to handle data sizes that are much larger than the size of main memory.

Disk Storage:

- Disk storage in DBMS is the permanent storage used to store large volumes of data efficiently.
- It ensures data persistence even after system shutdown.
- Data structures implemented by storage manager.

Disk storage Components:

- **Data files:** Stored in the database itself.
- **Data dictionary:** Stores metadata about the structure of the database.
- **Indices:** Provide fast access to data items.

6.What is ER model? Explain the basic symbols used for entities, attributes and relationships.(4)

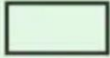
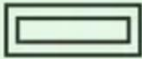
ANS:

- An entity relationship model describes relationship of entities that need to be stored in a database
- It is mainly a structured design for database
- It is a framework using specialized symbols to define the relationship between entities
- It is created based on three main components entities attributes and relationships
- ER diagrams require no technical knowledge of the underlying DBMS used.
- It gives a standard solution for visualizing the data logically.

Symbols Used in ER Model

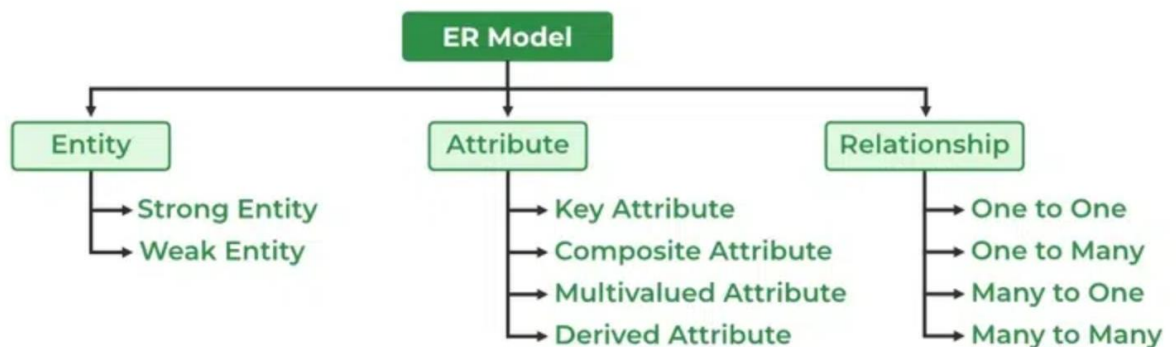
ER Model is used to model the logical view of the system from a data perspective which consists of these symbols:

- **Rectangles:** Rectangles represent Entities in the ER Model.
- **Ellipses:** Ellipses represent Attributes in the ER Model.
- **Diamond:** Diamonds represent Relationships among Entities.
- **Lines:** Lines represent attributes to entities and entity sets with other relationship types.
- **Double Ellipse:** Double Ellipses represent Multi-Valued Attributes.
- **Double Rectangle:** Double Rectangle represents a Weak Entity.

Figures	Symbols	Represents
Rectangle		Entities in ER Model
Ellipse		Attributes in ER Model
Diamond		Relationships among Entities
Line		Attributes to Entities and Entity Sets with Other Relationship Types
Double Ellipse		Multi-Valued Attributes
Double Rectangle		Weak Entity

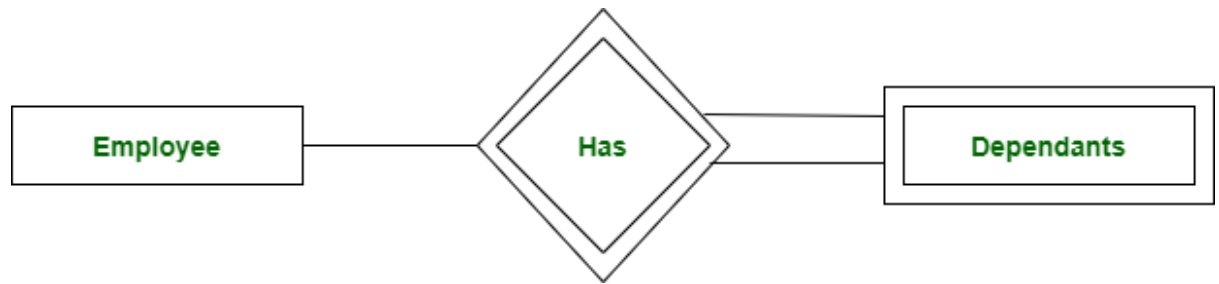
Components of ER Diagram

- ER Model consists of Entities, Attributes, and Relationships among Entities in a Database System.



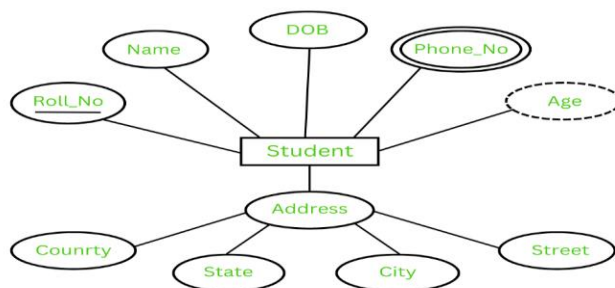
ENTITIES:

- An entity can be either living or non-living component
 - It is represented as rectangle in a ER diagram
- Strong entity:** It has a primary key that helps in identifying it uniquely, and it is represented by a rectangle and exists independently.
 - Weak entity:** A weak entity is an entity that cannot exist independently in a database and relies on a strong entity for its identification. A weak entity type is represented by a Double Rectangle.



ATTRIBUTES:

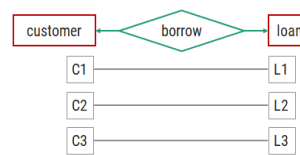
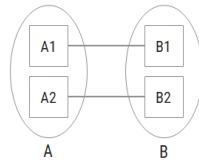
- An attribute describes the property of an entity
 - It is represented as Oval in ER diagram.
- Key Attribute:**
 - It uniquely identifies an entity from an entity set.
 - The text of key attributes underlined in ER diagram.
 - Composite Attribute:**
 - The attribute that is composed of other attributes is known as a composite attribute.
 - It is represented with an oval and that attribute is further connected with other ovals
 - Multi-Valued Attribute:**
 - Some attributes can compose over one value it is known as multi valued attributes
 - It is represented in double oval shape
 - Derived Attribute:**
 - An attribute that can be extracted from other attributes of an entity is known as derived attribute
 - It is represented by dashed oval.



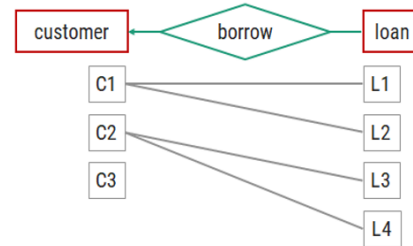
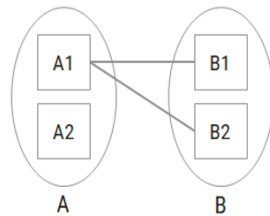
Relationships:

- The diamond shape is used for representing a relationship in ER diagram.
- It represents the relationship between two entities.

- **One -to-One relationship(1-1):** When an entity in A is associated with only one entity in B and an entity in B associated with only one entity in A then it is known as 1 to 1 relationship.

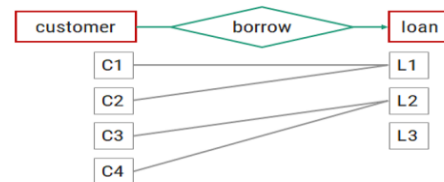
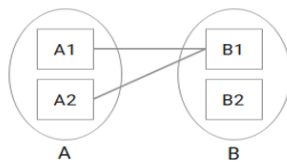


- **One -to-Many relationship(1-N):** When an entity in A is associated with more than one entities of B and an entity in B is associated with only one entity in A then it is known as one to many relationship.



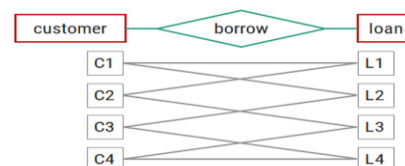
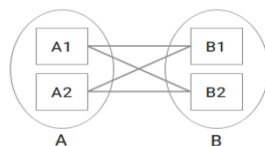
- **Many -to-One relationship(N-1):** An entity in A associated with only one entity in B and an entity in B is associated with more than one entities in then it is known as many to one relationship.

○

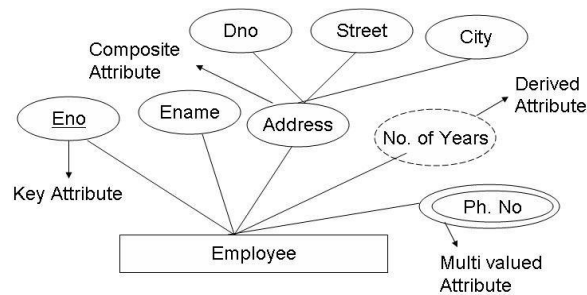


- **Many-to-Many Relationship(N-N):**An entity in A is associated with more than one entities in B and an Entity in B is associated with more than one entities in A.

○



7. What is an attribute? Explain attributes with various types of examples.(4)



attribute? Explain attributes with

- An attribute describes the property of an entity
- It is represented as Oval in ER diagram.
- i. **Key Attribute:**
 - It uniquely identifies an entity from an entity set.
 - The text of key attributes underlined in ER diagram.
 - **Eno (Employee Number)** → This uniquely identifies each employee.
- ii. **Composite Attribute:**
 - The attribute that is composed of other attributes is known as a composite attribute.
 - It is represented with an oval and that attribute is further connected with other ovals
 - **Address** → Made up of multiple sub-attributes:
 - **Dno (Door Number)**
 - **Street**
 - **City**
- iii. **Multi-Valued Attribute:**
 - Some attributes can compose over one value it is known as multi valued attributes
 - It is represented in double oval shape.
 - **Ph. No (Phone Number)** → An employee can have **multiple** phone numbers, represented with a **double oval**.
- iv. **Derived Attribute:**
 - An attribute that can be extracted from other attributes of an entity is known as derived attribute
 - It is represented by dashed oval.
 - **No. of Years** → This is calculated from other data, such as **joining date** or **birthdate**, and is represented with a dashed oval.
- v. **Simple (Atomic) Attribute:**
 - It contains a single, indivisible value for each entity.

- It ensures data is stored in its most basic form without further breakdown.
- **Ename (Employee Name)** → A single-valued attribute that cannot be divided further

8. What is Key? Distinguish between Super key, Candidate key, Primary Key, Foreign Key for a relation with examples.(4)

ANS:

- Keys are one of the basic requirements of a relational database model.
- It is widely used to identify the tuples(rows) uniquely in the table.
- We also use keys to set up relations amongst various columns and tables of a relational database.

Super key:

- A **super key** is a set of one or more attributes that uniquely identify a row (tuple) in a table.
- A super key **may contain extra attributes** that are not necessary for uniqueness.

Candidate Key:

- A **candidate** that uniquely table.
- A candidate **attributes** for ones from a

Example:

STUD_NO is the candidate key for relation STUDENT.

Table STUDENT

STUD_NO	SNAME	ADDRESS	PHONE
1	Shyam	Delhi	123456789
2	Rakesh	Kolkata	223365796
3	Suraj	Delhi	175468965

key is a **minimal** super key identifies each row (tuple) in a

key contains **only necessary** uniqueness (removing extra super key).

Primary Key:

- A **primary key** is that uniquely in a table.
- A primary key row and **cannot**

Table STUDENT

STUD_NO	SNAME	ADDRESS	PHONE
1	Shyam	Delhi	123456789
2	Rakesh	Kolkata	223365796
3	Suraj	Delhi	175468965

a **chosen candidate key** identifies each row (tuple)

must be unique for every **contain NULL values.**

Consider the table shown above.
STUD_NO+PHONE is a super key.

Example:

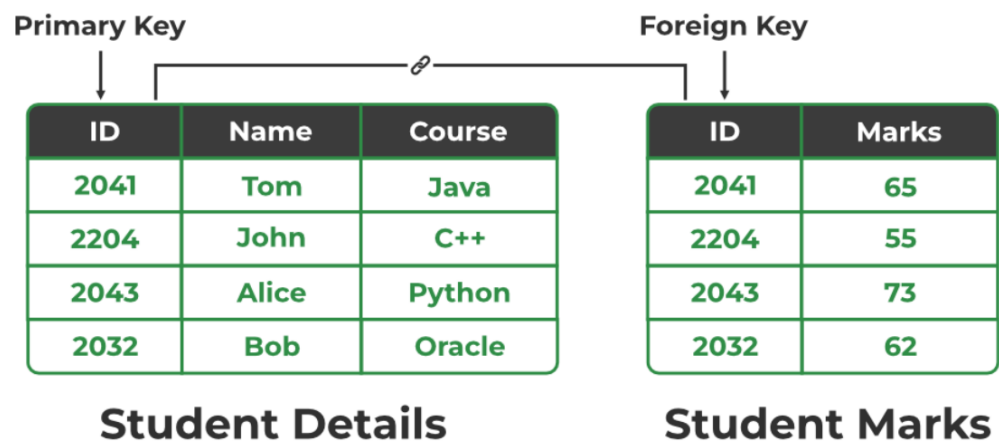
```
STUDENT table -> Student(STUD_NO, SNAME, ADDRESS, PHONE) , STUD_NO is a primary key
```

Table STUDENT

STUD_NO	SNAME	ADDRESS	PHONE
1	Shyam	Delhi	123456789
2	Rakesh	Kolkata	223365796
3	Suraj	Delhi	175468965

Foreign Key:

- A foreign key is an attribute in one table that refers to the primary key of another table, establishing a relationship.
- It helps connect two or more tables, enabling you to create relationships between them.



Example:

Student Table

Student_ID (PK)	Roll_Number (CK)	Name	Phone	Course_ID (FK)
101	2023001	John Doe	9876543210	CSE101
102	2023002	Alice Smith	8765432109	CSE102

Course Table

Course_ID (PK)	Course_Name
CSE101	Data Structures
CSE102	DBMS

Explanation

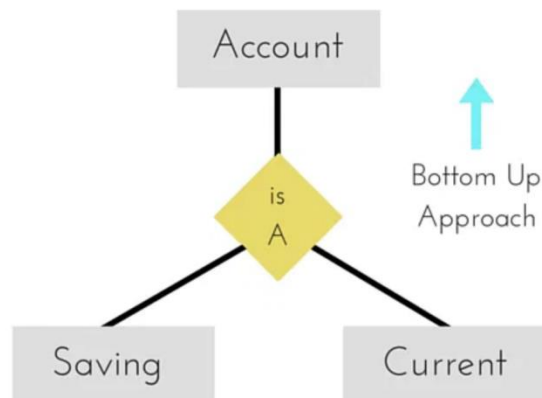
- Super Key: {Student_ID}, {Student_ID, Name}, {Roll_Number, Phone} (Can have extra attributes).
- Candidate Key: {Student_ID}, {Roll_Number} (Minimal unique attributes).
- Primary Key (PK): Student_ID (Chosen from candidate keys).
- Foreign Key (FK): Course_ID in the Student table links to Course_ID in the Course table.

9. How to represent generalization, specialization and aggregation using ER Diagrams. Explain with suitable example.(4)

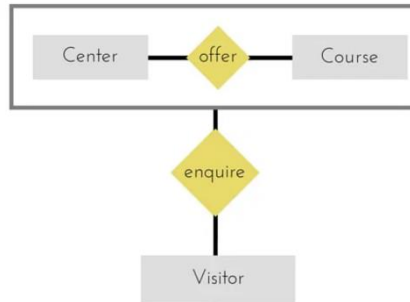
ANS:

Generalization:

- Generalization is a bottom-up approach in which two lower level entities combine to form a higher level entity.
- In generalization, the higher level entity can also combine with other lower level entities to make further higher level entity.
- For example, **Saving** and **Current** account types entities can be generalised and an entity with name **Account** can be created, which covers both.



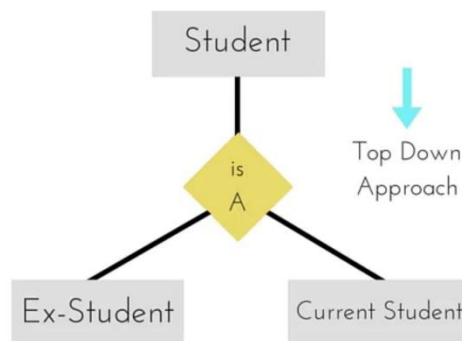
Specialization:



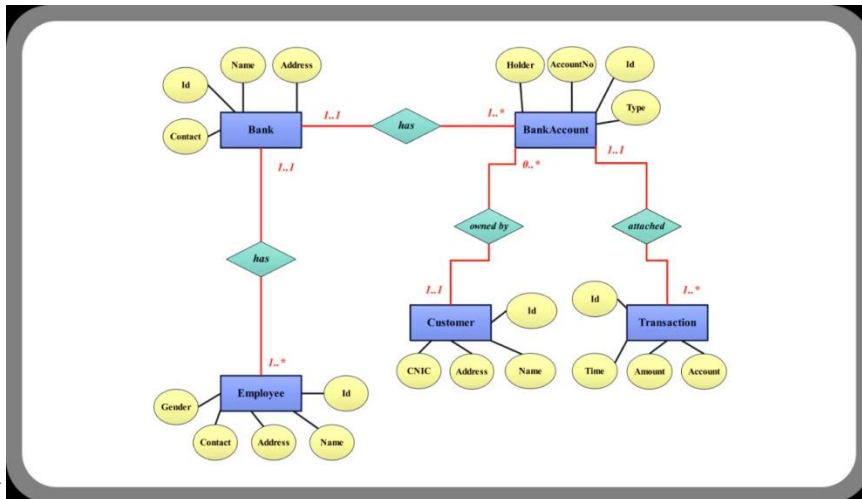
- Specialization is opposite to Generalization.
- It is a top-down approach in which one higher level entity can be broken down into two lower level entity.
- In specialization, a higher level entity may not have any lower-level entity sets, it's possible.
- For example, a **Student** can be specialized into **Current Student** and **Ex-Student**, where a **Current Student** is actively studying, while an **Ex-Student** has graduated or left the institution.

Aggregation:

- Aggregation is a process when relation between two entities is treated as a single entity.
- It is used when a relationship itself acts as an entity in another relationship.
- For Example, the relationship between **Center** and **Course** together, is acting as an Entity, which is in relationship with another entity **Visitor**.



10. Develop an E -R Diagram for Banking enterprise system.(8)



AN

[Refer entities, attributes, relationships from 6th and 7th long Answer]

Entities and Their Attributes:

➤ Bank

- Attributes: Id, Name, Address, Contact
- Each bank has a unique ID, name, address, and contact information.
- Relationship: A bank "has" multiple bank accounts.

➤ Bank Account

- Attributes: Id, Holder, AccountNo, Type
- Each bank account has an ID, account number, type (savings, checking, etc.), and a holder.
- Relationship:
 - A bank **has** multiple bank accounts (**1 to many**).
 - Each account **belongs to** a customer (**0 or more accounts per customer**).
 - A bank account **is attached to** multiple transactions (**1 to many**).

➤ Customer

- Attributes: Id, CNIC, Address, Name
- Each customer has a unique ID, CNIC (National Identification Number), address, and name.
- Relationship:
 - A customer **owns** a bank account (**1 to 1**).

➤ **Transaction**

- Attributes: Id, Time, Amount, Account
- Represents each transaction made within the bank, with an ID, timestamp, amount, and linked account.
- Relationship:
 - A **transaction is attached** to a bank account (**1 to many**).

➤ **Employee**

- Attributes: Id, Name, Address, Contact, Gender
- Each employee has a unique ID, name, address, contact, and gender.
- Relationship:
 - A bank **has** multiple employees (**1 to many**).

Relationships in the Diagram:

- Bank has BankAccount → (1 to many)
- Bank has Employees → (1 to many)
- Customer owns BankAccount → (1 to 1 or more)
- BankAccount is attached to Transaction → (1 to many)

11. Illustrate about integrity constraints with suitable examples

INTEGRITY CONSTRAINTS OVER RELATIONS

o Integrity constraints are a set of rules that are used to maintain the quality of information.

o Integrity constraints ensure that changes (update deletion, insertion) made to the database by authorized users do not result in a loss of data consistency.

There are many types of integrity constraints.

1. Entity Integrity Constraint/Key constraints
2. Referential Integrity constraints/ Foreign key
3. Domain constraints
4. General constraints

1. Entity Integrity Constraint/ Key constraints

The entity integrity constraint states that primary key value can't be null.

☐ This is because the primary key value is used to identify individual rows in relation and if the primary key has a null value, then we can't identify those rows.

□ A table can contain a null value other than the primary key field.

- **PRIMARY KEY:**

Primary key uniquely identifies each record in a table. It must have unique values and cannot contain nulls. In the below example the ROLL_NO field is marked as primary key, that means the ROLL_NO field cannot have duplicate and null values.

```
CREATE TABLE STUDENT( ROLL_NO INT NOT NULL,  
STU_NAME VARCHAR (35) NOT NULL UNIQUE, STU_AGE INT NOT NULL,  
STU_ADDRESS VARCHAR (35) UNIQUE,  
PRIMARY KEY (ROLL_NO)  
);
```

The UNIQUE keyword ensures that all values in a column are different.

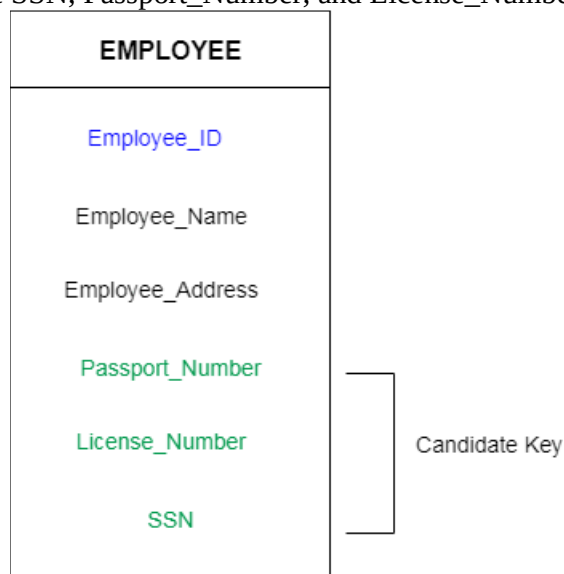
Specifying Primary Key Constraints in SQL

```
CREATE TABLE Students ( sid INT, name VARCHAR (30) ,Login VARCHAR(20) ,  
Age INT, gpa REAL, UNIQUE (name, age),  
CONSTRAINT StudentsKey PRIMARY KEY (sid) );
```

This definition says that *sid* is the primary key and the combination of *name* and *age* is also a key. The definition of the primary key also illustrates that a constraint should precede with **CONSTRAINT** *constraint-name*

- **CANDIDATE KEY**

- o The minimal set of attribute which can uniquely identify a tuple is known as candidate key.
- o They can contain null values. These are the subset of super key.
- o The remaining attributes except for primary key are considered as a candidate key.
- o **For example:** In the EMPLOYEE table, id is best suited for the primary key. Rest of the attributes like SSN, Passport_Number, and License_Number, etc. are considered as a candidate key.



- **SUPER KEY**

Super key is a (maximum) set of an attribute which can uniquely identify a tuple. Super key is a superset of a candidate key.

E.g: Let R be a relation with columns(A,B,C,D,E,F) such that by making AB as key we are able to identify all other columns i.e CDEF

Keys Is super key or not AB---- CDEF yes

CD----ABEF yes

CB----DF no

ABD---CEF yes

DF---ABCE yes

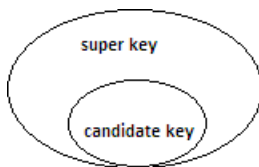
D---BC no

DEF---ABC yes

By making AB as key we are able to identify all other columns i.e CDEF. Similarly using CD,ABD,DF and DEF we are able to identify all. So all these are super key.

But by using CB we can only identify D and F. Hence CB is not a super key.

AB is a proper subset of super key ABD, when AB alone is able to identify all attributes, then we need not need ABD. Hence ABD is only super key and AB is candidate key.



NOTE: Each candidate key is super key but not vice versa.

2. Referential Integrity Constraint-

Foreign keys are the columns of a table that points to the primary key of another table. They act as a cross-reference between tables.

It states that if a foreign key exists in a relation then either the foreign key value must match a primary key value of some tuple in its home relation or the foreign key value must be null.

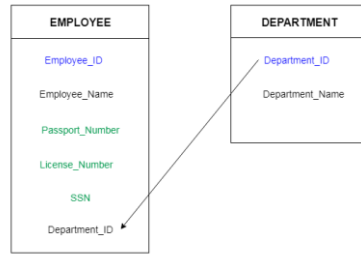
The rules are:

1. You can't delete a record from a primary table if matching records exist in a related table.
2. You can't change a primary key value in the primary table if that record has related records in foreign table.
3. You can't enter a value in the foreign key field of the related table that doesn't exist in the primary key of the primary table.
4. However, you can enter a Null value in the foreign key, specifying that the records are unrelated.

o E.g: In a company, every employee works in a specific department, and employee and department are two different entities. So we can't store the information of the department in the employee table. That's why we link these two tables through the primary key of one table.

o We add the primary key of the DEPARTMENT table, Department_Id as a new attribute in the EMPLOYEE table.

o Now in the EMPLOYEE table, Department_Id is the foreign key, and both the tables are related.



Specifying Foreign Key Constraints in SQL

```

CREATE TABLE Enrolled ( studid INT , cid INT,
grade CHAR(10),
PRIMARY KEY (studid, cid),
FOREIGN KEY (studid) REFERENCES Students)
  
```

The foreign key constraint states that every *studid* value in Enrolled must also appear in Students, that is, *studid* in Enrolled is a foreign key referencing Students. Specifically, every *studid* value in Enrolled must appear as the value in the primary key field, *sid*, of Students.

3. Domain constraint (Refer 3rd unit for detailed description)

A domain is a unique set of values permitted for an attribute in a table. For example, a domain of month-of-year can accept January....December as possible values, a domain of integers can accept whole numbers that are negative, positive and zero.

The data type of domain includes string, character, integer, time, date, currency, etc. The value of the attribute must be available in the corresponding domain.

Example:

ID	NAME	SEMENSTER	AGE
1000	Tom	1 st	17
1001	Johnson	2 nd	24
1002	Leonardo	5 th	21
1003	Kate	3 rd	19
1004	Morgan	8 th	A

Not allowed. Because AGE is an integer attribute

4. General Constraints

Sometimes specifying general constraints (like age should be greater than 18,etc) are necessary.

Current relational database systems support such general constraints in the form of *table constraints* and *assertions*. Table constraints are associated with a single table and checked whenever that table is modified. In contrast, assertions involve several tables and are checked whenever any of these tables is modified.

12.How to alter, destroy tables and views? Give example queries(4M)

ANS:1. ALTER TABLE (Modify Table Structure)

The ALTER TABLE statement is used to modify the structure of an existing table. You can:

- Add a new column
- Modify an existing column
- Rename a column
- Drop a column
- Rename the table

EXAMPLES:

(a) Add a New Column

```
sql
ALTER TABLE employees ADD department VARCHAR(50);
```

- ◆ Adds a new column `department` of type `VARCHAR(50)` to the `employees` table.

(b) Modify an Existing Column

```
sql
ALTER TABLE employees MODIFY department TEXT;
```

- ◆ Changes the `department` column data type from `VARCHAR(50)` to `TEXT`.

(c) Rename a Column

```
sql
ALTER TABLE employees RENAME COLUMN department TO dept;
```

- ◆ Renames the column `department` to `dept`.

2. DROP TABLE (Destroy a Table)

The DROP TABLE statement permanently removes a table and its data.

Example:

```
sql
DROP TABLE employees;
```

- ⚠ Warning: This deletes the table and all its records permanently. It cannot be undone.

3. ALTER VIEW (Modify an Existing View)

The ALTER VIEW statement modifies an existing view without dropping it.

Example:

```
ALTER VIEW employee_view AS
```

```
SELECT id, name, salary FROM employees WHERE salary > 50000;
```

◆ Modifies the employee_view to show only employees with a salary above 50,000.

4.DROP VIEW (Destroy a View)

The DROP VIEW statement **removes a view definition**.

Example:

```
DROP VIEW employee_view;
```

◆ Deletes the employee_view, but the base table remains unchanged.

13.Explain Views and its merits &demerits.(4M)

ANS:

- ❖ A view is virtual tables whose rows are not explicitly stored in the database.
- ❖ They are used to restrict access to the database or to hide data complexity.
- ❖ A view contains rows and columns, just like a real table.
- ❖ Creating a view does not take any storage space as only the view query is stored in the data dictionary and the actual data is not stored.
- ❖ The tables referred in the views are known as Base tables.
- ❖ Views do not contain data of their own.
- ❖ They take data from the base tables

Merits of Views:

Simplified Queries: Views simplify complex queries into a single, user-friendly entity.

Enhanced Security: Restricts access to specific data, improving security.

Data Independence: Changes in base tables don't always require modifying views.

Data Abstraction: Users interact with a simplified version of data, hiding schema complexity.

Demerits of Views:

Performance Issues: Queries on views may be slower than direct table queries.

Update Complexity: Updating data through views can be tricky due to structural constraints.

Table Dependency: If base tables are modified or dropped, views may break.

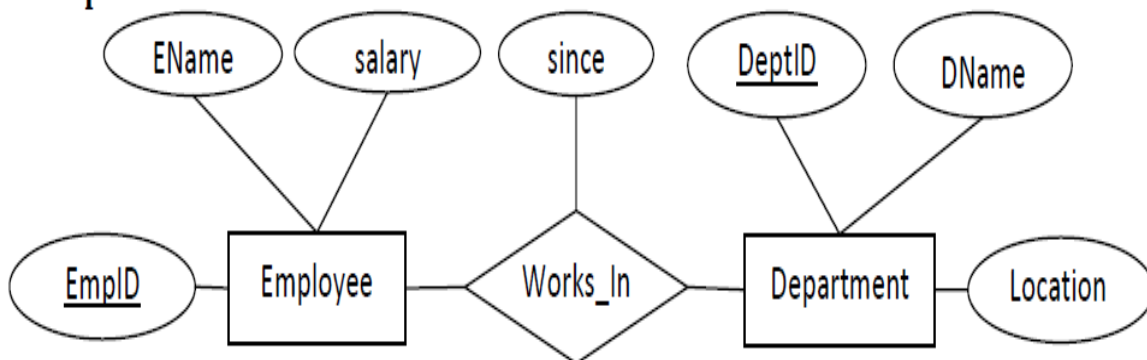
Memory Consumption: Large views can consume more memory, affecting performance.

14.Elaborate on logical database design with examples.(4M)

ANS:

1. Each entity in the ER model will become a table and all attributes of that entity will become columns of the table. **Key attribute of the entity will become primary key in the table.**

Example:



Employee		
EmpID	EName	salary

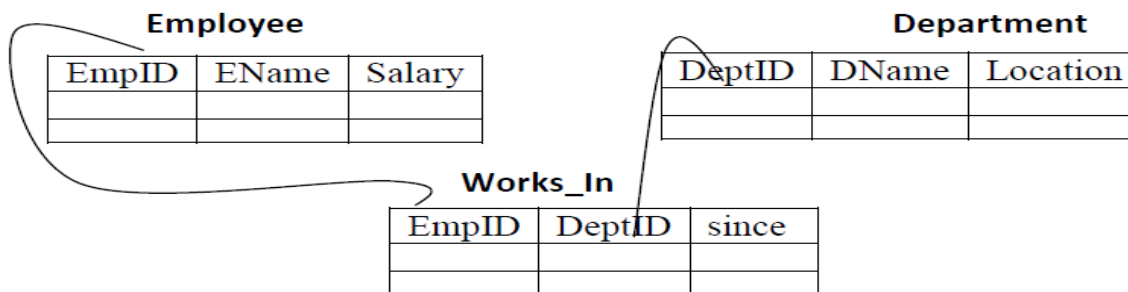
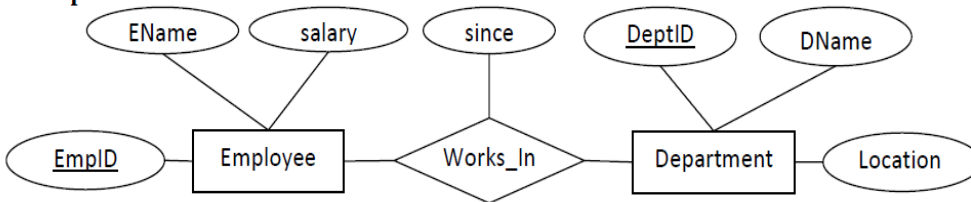
Department		
DeptID	DName	Location

```
CREATE TABLE Employee
(
  EmpID NUMBER(3),
  EName VARCHAR(20),
  Salary  NUMBER(5),
  PRIMARY KEY(EmpID)
);
```

```
CREATE TABLE Department
(
  DeptID NUMBER(3),
  DName VARCHAR(15),
  Location VARCHAR(15),
  PRIMARY KEY(DeptID)
);
```

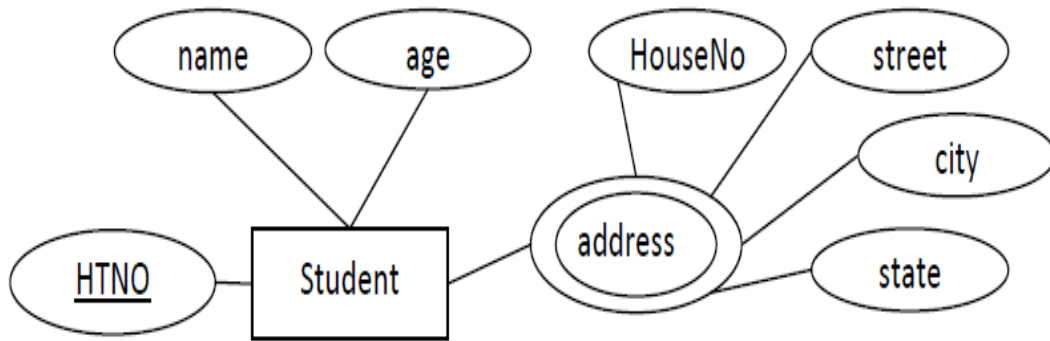
2. Each relationship in the ER model will become a table. Key attributes of participating entities in the relationship will become columns of the table. If the relationship has any attributes, then they also will become columns of the table.

Example:



```
CREATE TABLE Works_In
(
  EmpID NUMBER(3),
  DeptID NUMBER(3),
  Since  DATE,
  PRIMARY KEY(EmpID, DeptID),
  FOREIGN KEY (EmpID) REFERENCES Employee(EmpID),
  FOREIGN KEY (DeptID) REFERENCES Department(DeptID),
);
```

3. Any multi-valued attribute is converted into new table. The primary key of the entity will be added as column in the new table.



Student

HTNO	name	age

Address

HTNO	houseNo	street	city	state

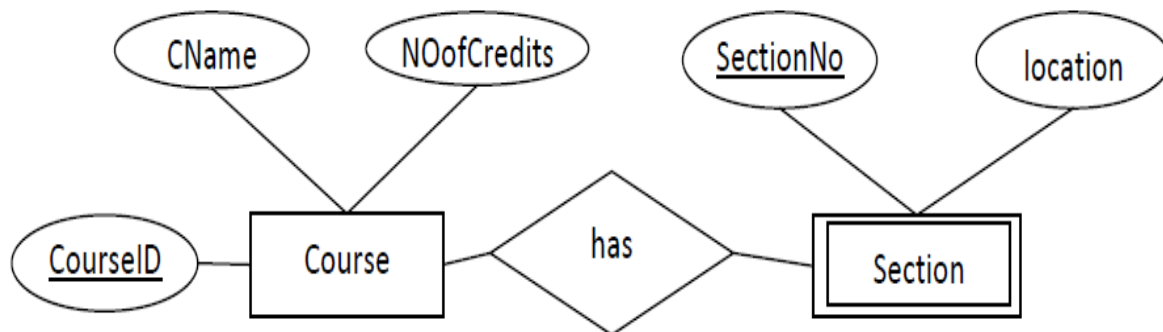
```

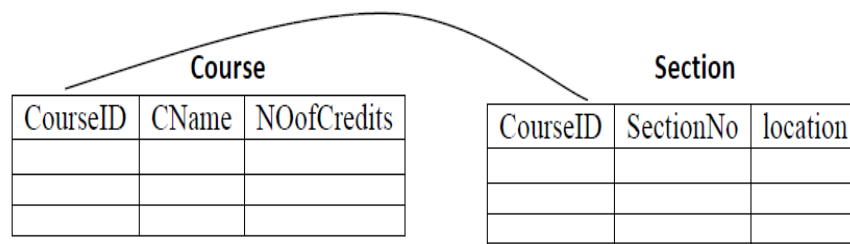
CREATE TABLE Student
(
    HTNO CHAR(10),
    name VARCHAR(20),
    age NUMBER(2),
    PRIMARY KEY(HTNO)
);
  
```

```

CREATE TABLE Address
(
    HTNO CHAR(10),
    houseNo NUMBER(3),
    street VARCHAR(20),
    city VARCHAR(15),
    state VARCHAR(15),
    PRIMARY KEY(HTNO),
    FOREIGN KEY (HTNO) REFERENCES Student(HTNO),
);
  
```

4. Each *weak entity* is converted into a table with all its attributes as columns and *primary key of the strong entity acts as a foreign key in this table*.





```
CREATE TABLE Course
```

```
(
```

```
    CourseID NUMBER(2),
    CName  VARCHAR(20),
    NOofCredits NUMBER(2),
    PRIMARY KEY(CourseID)
```

```
);
```

```
CREATE TABLE Section
```

```
(
```

```
    SectionNo CHAR(2),
    CourseID NUMBER(2),
    location  VARCHAR(15),
    PRIMARY KEY(CourseID, SectionNo),
    FOREIGN KEY(CourseID) REFERENCES Course(CourseID)
```

```
);
```

15.Explain the fundamental operations in relational algebra with examples.(8M).

RELATIONAL ALGEBRA

Relational Algebra is procedural query language, which takes Relation as input and generates relation as output. Relational algebra mainly provides theoretical foundation for relational databases and SQL.

Operator Symbol	Operator Name	Explanation
π	Projection	Select column names
σ	Selection	Select row values
ρ	Renaming	Rename a table name or expression results
$\cup$	Union	Perform union operation
$\cap$	Intersection	Perform intersection operation
$-$	Set deference	Perform set difference operation
$\times$	Cartesian product	Every row of first table is joined with every row of second table
$\bowtie$	Join	Join two tables based on some condition

1.Select Operation (σ):It selects tuples that satisfy the given predicate from a relation.

Notation : $\sigma_p(r)$

- where σ stands for selecting tuples (rows) and r stands for relation (table) name.
- p is propositional logic formula which may use connectors like **and**, **or**, and **not**.

Example : $\sigma_{\text{subject} = \text{"database"}}(\text{Books})$

Table: Books

BookID	Title	Author	Subject	Year
1	Database Fundamentals	John Doe	Database	2020
2	Data Science Essentials	Jane Smith	Data Science	2021
3	SQL for Beginners	Alice Brown	Database	2022
4	Machine Learning Basics	Bob White	Machine Learning	2023

SQL Query:

```
SELECT * FROM Books WHERE Subject = 'database';
```

Output:

BookID	Title	Author	Subject	Year
1	Database Fundamentals	John Doe	Database	2020
3	SQL for Beginners	Alice Brown	Database	2022

This query selects rows from the "Books" table where the **Subject** column equals 'database'.

2.Project Operation (Π):It projects column(s) that satisfy a given predicate.

Notation: $\Pi_{A1, A2, \dots, An}(r)$

where $A1, A2, \dots, An$ are column (attribute) names of relation r . Duplicate rows are automatically eliminated in the output.

Example: $\Pi_{\text{subject}, \text{author}}(\text{Books})$

Table: Books

BookID	Title	Author	Subject	Year
1	Database Fundamentals	John Doe	Database	2020
2	Data Science Essentials	Jane Smith	Data Science	2021
3	SQL for Beginners	Alice Brown	Database	2022
4	Machine Learning Basics	Bob White	Machine Learning	2023

SQL Query: SELECT Subject, Author FROM Books;

Subject	Author
Database	John Doe
Data Science	Jane Smith
Database	Alice Brown
Machine Learning	Bob White

In this query, we are projecting only the Subject and Author columns from the Books table.

3.Union Operation (U):It performs union operation between two given relations. It combines rows from two given relations.

Notation: $r \cup s$

Example: $\Pi \text{ author(Books)} \cup \Pi \text{ author(Articles)}$

Table 1: Books

BookID	Title	Author	Subject	Year
1	Database Fundamentals	John Doe	Database	2020
2	Data Science Essentials	Jane Smith	Data Science	2021
3	SQL for Beginners	Alice Brown	Database	2022

Table 2: Articles

ArticleID	Title	Author	Topic	Year
1	Introduction to Databases	Mary Johnson	Databases	2022
2	Machine Learning Overview	Bob White	Machine Learning	2023
3	Deep Dive into AI	John Doe	Artificial Intelligence	2021

Result of $\Pi \text{ author(Books)} \cup \Pi \text{ author(Articles)}$:

Author
John Doe
Jane Smith
Alice Brown
Mary Johnson
Bob White

4.Intersection Operation ($\cap$):It performs intersection operation between two given relations. It collect only rows which are **common in the two given relations**.

Notation: $R \cap S$

$R \cap S$ returns a relation containing all tuples that occur in *both* R and S .

Example: $\Pi \text{ author(Books)} \cap \Pi \text{ author(Articles)}$

Table 1: Books

BookID	Title	Author	Subject	Year
1	Database Fundamentals	John Doe	Database	2020
2	Data Science Essentials	Jane Smith	Data Science	2021
3	SQL for Beginners	Alice Brown	Database	2022

Table 2: Articles

ArticleID	Title	Author	Topic	Year
1	Introduction to Databases	Mary Johnson	Databases	2022
2	Machine Learning Overview	Bob White	Machine Learning	2023
3	Deep Dive into AI	John Doe	Artificial Intelligence	2021

Result of $\Pi \text{ author(Books)} \cap \Pi \text{ author(Articles)}$:

Author
John Doe

5.Set Difference (-):It finds tuples which are present in one relation but not in the second relation.

Notation: $r - s$

Finds all the tuples that are **present in r but not in s**

Example: $\Pi \text{ author (Books)} - \Pi \text{ author (Articles)}$

Table 1: Books

BookID	Title	Author	Subject	Year
1	Database Fundamentals	John Doe	Database	2020
2	Data Science Essentials	Jane Smith	Data Science	2021
3	SQL for Beginners	Alice Brown	Database	2022

Table 2: Articles

ArticleID	Title	Author	Topic	Year
1	Introduction to Databases	Mary Johnson	Databases	2022
2	Machine Learning Overview	Bob White	Machine Learning	2023
3	Deep Dive into AI	John Doe	Artificial Intelligence	2021

Result of $\Pi \text{ author (Books)} - \Pi \text{ author (Articles)}$:

Author
Jane Smith
Alice Brown

These authors appear in the **Books** table but not in the **Articles** table.

6.Rename Operation (ρ):

- ❖ The results of relational algebra are also relations but without any name.
- ❖ The rename operation allows us to rename the output relation.
- ❖ 'rename' operation is denoted with small Greek letter rho ρ .

Notation: $\rho(\text{temp}, E)$

Where the result of expression **E** is saved with name of **temp**.

7.Cartesian Product (X):It returns a relation whose schema contains all the fields of table-1 (in the same order as they appear in table-1) followed by all the fields of table-2.

It combines every row in first table with every row in the second table.

Notation: $r \times s$

Where r and s are relations and their output will be defined as :

$$r \times s = \{ q \mid q \in r \text{ and } t \in s \}$$

Example:

Let's define two relations r and s .

Relation r (Employee):

EmployeeID	Name	Department
1	John Doe	HR
2	Alice Smith	IT

Relation s (Project):

ProjectID	ProjectName
101	Database Upgrade
102	Website Redesign

Cartesian Product $r \times s$:

When we perform $r \times s$, we combine each row from r with each row from s . So, for every employee in r , we pair them with every project in s .

Result of $r \times s$:

EmployeeID	Name	Department	ProjectID	ProjectName
1	John Doe	HR	101	Database Upgrade
1	John Doe	HR	102	Website Redesign
2	Alice Smith	IT	101	Database Upgrade
2	Alice Smith	IT	102	Website Redesign

8.Division (/):

- Consider two relation A and B in which A has (exactly) two fields x and y and B has just one field y , with the same domain as in A .
- We define the *division* operation A / B as the set of all x values (in the form of unary tuples) such that for every y value in (a tuple of) B , there is a tuple (x,y) in A .
- $A \subseteq B$

Example:

<i>A</i>	SNO	PNO	<i>B1</i>	PNO	<i>A / B1</i>	SNO
	S1	P1		P2		S1
	S1	P2	<i>B2</i>	PNO		S2
	S1	P3		P2		S3
	S1	P4		P4		S4
	S2	P1	<i>B3</i>	PNO	<i>A / B2</i>	SNO
	S2	P2		P2		S1
	S3	P2		P4	<i>A / B3</i>	S4
	S4	P2				SNO
	S4	P4				S1

9.

join ($\bowtie$):

The *join condition* is identical to a *selection condition* in form. The operation is defined as follows:

$$R \bowtie_c S = \sigma_c(R \times S)$$

Thus $\bowtie$ is defined to be a cross-product followed by a selection.

Note that the condition c can refer to attributes of both R and S . $R \bowtie_c S$

16.Explain Tuple relational calculus.(4M)

ANS:RELATIONAL CALCULUS

- ❖ Relational calculus is an alternative to relational algebra.
- ❖ In contrast to the algebra, which is procedural, the calculus is nonprocedural, or *declarative*.

- ❖ It allows us to describe the set of answers without being explicit about how they should be computed.

Tuple Relational Calculus

- ❖ Tuple Relational Calculus is a non-procedural query language unlike relational algebra.
- ❖ Tuple Calculus provides only the description of the query but it does not provide the methods to solve it.
- ❖ Thus, it explains what to do but not how to do.

In Tuple Relational Calculus,

a query is expressed as $\{t \mid P(t)\}$

Where,

t = resulting tuples,

$P(t)$ = known as Predicate and these are the conditions that are used to fetch t .

Thus, it generates set of all tuples t , such that Predicate $P(t)$ is true for t .

$P(t)$ may have various conditions logically combined with

OR ($\vee$), AND ($\wedge$), NOT ($\neg$).

It also uses quantifiers: $\exists t \in r (Q(t))$

“there exists” a tuple in t in relation r such that predicate $Q(t)$ is true.

$\forall t \in r (Q(t))$

$Q(t)$ is true “for all” tuples in relation r .

Example - Student Database Schema:

Let's assume we have a **Student** relation with the following attributes:

Student(ID, Name, Age, Major, GPA)

Example 1:

Find all students majoring in 'Computer Science'

$\{S \mid S \in Student \wedge S.Major = 'ComputerScience'\}$

This expression returns all tuples **S** from the **Student** relation where the **Major** attribute is 'Computer Science'.

17. Discuss briefly about Domain relational calculus with suitable example. (4M)

ANS:

- ❖ Domain Relational Calculus (DRC) is a non-procedural query language used in relational databases.
- ❖ Unlike SQL, which specifies *how* to retrieve data, DRC focuses on *what* data to retrieve using domain variables.
- ❖ In DRC, queries are written using logical expressions to specify conditions on the attributes (columns) of a relation (table).

Syntax of DRC

A DRC query is expressed as:

$\{ \langle X_1, X_2, \dots, X_n \rangle \mid \text{Condition}(X_1, X_2, \dots, X_n) \}$

- $\{ \}$ → Defines the **result set**.
- $\langle X_1, X_2, \dots, X_n \rangle$ → Represents **domain variables** (values from columns).
- $\text{Condition}(X_1, X_2, \dots, X_n)$ → Specifies the **filtering condition**.

Example Database

Let's consider a **Student Table**:

Roll_No	Name	Age	Course
101	Alice	21	DBMS
102	Bob	22	OOP
103	Charlie	23	Networks
104	David	21	DBMS

Example Queries in DRC

1. Find Names of Students Enrolled in "DBMS"

$\{ \langle X \rangle \mid \exists Y \exists Z (\text{Student}(X, Y, Z, \text{"DBMS"})) \}$

Explanation:

- $\langle X \rangle$ → The result should contain **Names**.
- $\exists Y \exists Z$ → There exist **some Age (Y) and Roll_No (Z)**.

- Student(X, Y, Z, "DBMS") → Retrieves rows where the **Course is "DBMS"**.

Output

Name
Alice
David

18.Explain Enforcing integrity Constraints over a Relation.(4M)

ICs are specified when a relation is created and **enforced** or **imposed** when a relation is modified. The impact of domain, PRIMARY KEY, and UNIQUE constraints is

<i>sid</i>	<i>name</i>	<i>login</i>	<i>age</i>	<i>gpa</i>
50000	Dave	dave@cs	19	3.2
53666	Jones	jones@cs	18	3.3
53688	Smith	smith@ee	18	3.2
53650	Smith	smith@math	19	3.7
53831	Madayan	madayan@music	11	1.8
53832	Guldu	guldu@music	12	2.0

straightforward. If an insert, delete, or update command causes a violation, it is rejected. Potential IC violation is generally checked at the end of each SQL statement execution, although it can be deferred until the end of the transaction executing the statement. Consider the instance Students shown in above table. The following insertion violates the primary key constraint because there is already a tuple with the sid 53688, and it will be rejected by the DBMS.

```
INSERT
INTO Students (sid, name, login, age, gpa)
VALUES (53688, 'Mike', 'mike@ee', 17, 3.4)
```

The following insertion violates the constraint that the primary key cannot contain null.
INSERT

```
INTO Students (sid, name, login, age, gpa)
VALUES (null, 'Mike', 'mike@ee', 17, 3.4)
```

A similar problem arises whenever we try to insert a tuple with a value in a field that is not in the domain associated with that field, i.e., whenever we violate a domain constraint. Deletion does not cause a violation of domain, primary key or unique constraints. However, an update can cause violations, similar to an insertion.

```
UPDATE Students S
SET S.sid = 50000
WHERE S.sid = 53688
```

This update violates the primary key constraint because there is already a tuple with sid 50000.

The impact of foreign key constraints is more complex because SQL sometimes tries to rectify a foreign key constraint violation instead of simply rejecting the change. SQL provides several alternative ways to handle foreign key violations. We must consider three basic questions:

1. What should we do if an Enrolled row is inserted, with a sid column value that does not appear in any row of the Students table?
2. What should we do if a Students row is deleted?
3. What should we do if the primary key value of a Students row is updated?

The options are:

- ☐ ☐ Delete all Enrolled rows that refer to the deleted Students row.
- ☐ ☐ Disallow the deletion of the Students row if an Enrolled row refers to it.
- ☐ ☐ Set the sid column to the sid of some (existing) 'default' student, for every Enrolled row that refers to the deleted Students row.
- ☐ ☐ For every Enrolled row that refers to it, set the sid column to null. In our example, this option conflicts with the fact that sid is part of the primary key of Enrolled and therefore cannot be set to null. Thus, we are limited to the first three options in our example, although this fourth option is setting the foreign key to null is available in the general case.

The options here are similar to the previous case.

SQL allows us to choose any of the four options on DELETE and UPDATE. For example, we can specify that when a Students row is deleted, all Enrolled rows that refer to it are to be deleted as well, but that when the sid column of a Students row is modified, this update is to be rejected if an Enrolled row refers to the modified Students row.

```
CREATE TABLE Enrolled (sid CHAR(20),
cid CHAR(20),
grade CHAR(10),
PRIMARY KEY (sid, cid),
FOREIGN KEY (sid) REFERENCES Students
ON DELETE CASCADE
ON UPDATE NO ACTION)
```

The options are specified as part of the foreign key declaration. The default option is NO ACTION, which means that the action DELETE or UPDATE is to be rejected. Thus, the ON UPDATE clause in our example could be omitted, with the same effect. The CASCADE keyword says that if a Students row is deleted, all Enrolled rows that refer to it are to be deleted as well. If the UPDATE clause specified CASCADE, and the sid column of a Students row is updated, this update is also carried out in each Enrolled row that refers to the

updated Students row.

If a Students row is deleted, we can switch the enrollment to a 'default' student by using ON DELETE SET DEFAULT. The default student is specified as part of the definition of the sid field in Enrolled; for example, sid CHAR(20) DEFAULT '53666'. Although the specification of a default value is appropriate in some situations (e.g., a default parts supplier if a particular supplier goes out of business), it is really not appropriate to switch enrollments to a default student. The correct solution in this example is to also delete all enrollment tuples for the deleted student (that is, CASCADE), or to reject the update.

SQL also allows the use of null as the default value by specifying ON DELETE SET NULL.

ON DELETE CASCADE: Means when a row is deleted from the base relation, then all the rows which referred to this deleted row in the dependent relation must also be deleted.

ON UPDATE CASCADE: Means when updations are carried in base relation for the primary key attribute, then all this updations must also be carried out in the dependent relation or foreign key table.

ON DELETE SET DEFAULT: It means when a row is deleted in employee, then that row in dependent relation can be set to same default value.

ON DELETE SET NULL: It means on deleting the row in base relation the same row can be assigned a NULL value in dependent relation.

SQL even provides the facility to delay the applications of constraints on relation and also immediate application of constraints. This is possible with these two additional Constraints.

1. DEFERRED Mode

2. IMMEDIATE Mode

The syntax for this constraint is

SET CONSTRAINT constraint name DEFERRED.

SET CONSTRAINT constraint name IMMEDIATE.

Usually, constraints are checked at the end of SQL statements and if the constraints are violated then the statements are rejected. But with DEFERRED constraints, the Constraints checks are postponed and are checked at the time of commit.

19. Let $R = (ABC)$ and $S = (DEF)$ let $r(R)$ and $s(S)$ be relations on schema R and S . Give an expression in the Domain relational calculus that is equivalent to each of the following. i) $\sigma_{B=25}(r)$ ii) $\prod[A, F, (\sigma_{C=D}(r \times s))$. (4M)

Let's break down your request into Domain Relational Calculus expressions:

i) $\sigma_{B=25}(r)$

This expression in relational algebra represents a selection where the value of attribute **B** is equal to **25** in the relation **r** (with schema **R = (ABC)**).

In Domain Relational Calculus, the equivalent expression would be:

$\{t \mid \exists a, b, c (t = (a, b, c) \wedge b = 25 \wedge (a, b, c) \in r)\} \setminus \{t \mid \exists a, b, c, (t = (a, b, c) \wedge b = 25 \wedge \langle a, b, c \rangle \in r)\}$

Here:

- t represents a tuple from the relation.
- a, b, c, a, b, c are the domain variables for attributes **A**, **B**, and **C** respectively.
- We are selecting tuples where the value of **B** is 25.

ii) $\pi_{A,F}(\sigma_{C=D}(r \times s))$

This expression represents a projection of attributes **A** and **F** after performing a selection on the Cartesian product of relations **r** and **s**, where **C = D**.

In Domain Relational Calculus, the equivalent expression would be:

$\{(a,f) \mid \exists b,c,d,e (\langle a,b,c \rangle \in r \wedge \langle d,e,f \rangle \in s \wedge c=d)\} \setminus \{(a,f) \mid \exists b,c,d,e (\langle a,b,c \rangle \in r \wedge \langle d,e,f \rangle \in s \wedge c=d)\}$

Here:

- (a,f) are the projected attributes, representing the values of **A** and **F**.
- $b,c,d,e, c, d, e, c, d, e$ represent the domain variables for attributes **B**, **C** (in relation **r**), and **D**, **E** (in relation **s**), respectively.
- The condition $c = d$ filters the Cartesian product such that the values of **C** and **D** match.

So, for the second part, you first select the tuples where **C = D** in the Cartesian product, then project only the **A** and **F** values.

20. Consider the following relations Sailors (sid, sname, rating, age) Boats (bid, bname, color) Reserves (sid, bid, day) Write the statements in Relational Algebra, Relational Calculus, Domain Relational Calculus and SQL for the following questions. a) Find the names of sailors who have reserved a Red boat. b) Find the names of sailors who have reserved at least one boat. c) Find the names of sailors who have reserved a Red and a Green boat. d) Find the names of sailors who have reserved a Red or a White boat. e) Find the names of sailors who have reserved all boats(8M)

ANS: Here are the queries for Relational Algebra, Tuple Relational Calculus (TRC), Domain Relational Calculus (DRC), and SQL based on the given relations:

Given Relations

1. Sailors (sid, sname, rating, age)
2. Boats (bid, bname, color)
3. Reserves (sid, bid, day)

(a) Find the names of sailors who have reserved a Red boat.

Relational Algebra

$\pi_{\text{sname}} (\sigma_{\text{color}='Red'} (\text{Boats}) \bowtie \text{Reserves} \bowtie \text{Sailors})$

- Join **Boats** and **Reserves** on **bid**.
- Join the result with **Sailors** on **sid**.
- Select tuples where **color** = 'Red'.
- Project **sname**.

Tuple Relational Calculus (TRC)

$\{S.\text{sname} \mid \exists B \exists R (\text{Sailors}(S) \wedge \text{Reserves}(R) \wedge \text{Boats}(B) \wedge S.\text{sid} = R.\text{sid} \wedge R.\text{bid} = B.\text{bid} \wedge B.\text{color} = \text{'Red'}) \}$

Domain Relational Calculus (DRC)

$\{ \langle N \rangle \mid \exists I \exists R \exists A \exists J \exists B \exists C (\text{Sailors}(I, N, R, A) \wedge \text{Reserves}(I, J, D) \wedge \text{Boats}(J, B, C) \wedge C = \text{'Red'}) \}$

SQL Query

SELECT DISTINCT S.sname

FROM Sailors S

JOIN Reserves R ON S.sid = R.sid

JOIN Boats B ON R.bid = B.bid

WHERE B.color = 'Red';

(b) Find the names of sailors who have reserved at least one boat.

Relational Algebra

$\pi_{\text{sname}} (\text{Sailors} \bowtie \text{Reserves})$

- Join **Sailors** and **Reserves** on **sid**.
- Project **sname**.

Tuple Relational Calculus (TRC)

$\{ S.sname \mid \exists R (Sailors(S) \wedge Reserves(R) \wedge S.sid = R.sid) \}$

Domain Relational Calculus (DRC)

$\{ \langle N \rangle \mid \exists I \exists R \exists A \exists J (Sailors(I, N, R, A) \wedge Reserves(I, J, D)) \}$

SQL Query

SELECT DISTINCT S.sname

FROM Sailors S

JOIN Reserves R ON S.sid = R.sid;

(c) Find the names of sailors who have reserved a Red and a Green boat.

Relational Algebra

$\pi_{sname} (\sigma_{color='Red'} (Boats) \bowtie Reserves \bowtie Sailors) \cap \pi_{sname} (\sigma_{color='Green'} (Boats) \bowtie Reserves \bowtie Sailors)$

Tuple Relational Calculus (TRC)

$\{ S.sname \mid \exists B1 \exists B2 \exists R1 \exists R2 (Sailors(S) \wedge Reserves(R1) \wedge Reserves(R2) \wedge Boats(B1) \wedge Boats(B2) \wedge S.sid = R1.sid \wedge S.sid = R2.sid \wedge R1.bid = B1.bid \wedge R2.bid = B2.bid \wedge B1.color = 'Red' \wedge B2.color = 'Green') \}$

Domain Relational Calculus (DRC)

$\{ \langle N \rangle \mid \exists I \exists R1 \exists R2 \exists J1 \exists J2 \exists B1 \exists B2 \exists C1 \exists C2 (Sailors(I, N, R1, A) \wedge Reserves(I, J1, D1) \wedge Reserves(I, J2, D2) \wedge Boats(J1, B1, C1) \wedge Boats(J2, B2, C2) \wedge C1 = 'Red' \wedge C2 = 'Green') \}$

SQL Query

SELECT DISTINCT S.sname

FROM Sailors S

JOIN Reserves R1 ON S.sid = R1.sid

JOIN Boats B1 ON R1.bid = B1.bid

JOIN Reserves R2 ON S.sid = R2.sid

JOIN Boats B2 ON R2.bid = B2.bid
WHERE B1.color = 'Red' AND B2.color = 'Green';

(d) Find the names of sailors who have reserved a Red or a White boat.

Relational Algebra

$\pi_{\text{sname}} (\sigma_{\text{color}='Red'} (\text{Boats}) \bowtie \text{Reserves} \bowtie \text{Sailors}) \cup \pi_{\text{sname}} (\sigma_{\text{color}='White'} (\text{Boats}) \bowtie \text{Reserves} \bowtie \text{Sailors})$

Tuple Relational Calculus (TRC)

$\{ S.\text{sname} \mid \exists B \exists R (\text{Sailors}(S) \wedge \text{Reserves}(R) \wedge \text{Boats}(B) \wedge S.\text{sid} = R.\text{sid} \wedge R.\text{bid} = B.\text{bid} \wedge (B.\text{color} = 'Red' \vee B.\text{color} = 'White')) \}$

Domain Relational Calculus (DRC)

$\{ \langle N \rangle \mid \exists I \exists R \exists A \exists J \exists B \exists C (\text{Sailors}(I, N, R, A) \wedge \text{Reserves}(I, J, D) \wedge \text{Boats}(J, B, C) \wedge (C = 'Red' \vee C = 'White')) \}$

SQL Query

```
SELECT DISTINCT S.sname
FROM Sailors S
JOIN Reserves R ON S.sid = R.sid
JOIN Boats B ON R.bid = B.bid
WHERE B.color = 'Red' OR B.color = 'White';
```

(e) Find the names of sailors who have reserved all boats.

Relational Algebra

$\pi_{\text{sname}} (\text{Sailors}) - \pi_{\text{sname}} ((\text{Sailors} \times \text{Boats}) - \text{Reserves})$

- Find sailors who have **not reserved any boat**.
- Subtract them from **all sailors**.

Tuple Relational Calculus (TRC)

$\{ S.sname \mid \forall B (Boats(B) \rightarrow \exists R (Reserves(R) \wedge S.sid = R.sid \wedge R.bid = B.bid)) \}$

Domain Relational Calculus (DRC)

$\{ \langle N \rangle \mid \exists I \exists R \exists A (Sailors(I, N, R, A) \wedge \forall J \exists D (Boats(J, B, C) \rightarrow Reserves(I, J, D))) \}$

SQL Query

SELECT DISTINCT S.sname

FROM Sailors S

WHERE NOT EXISTS (

 SELECT B.bid FROM Boats B

 WHERE NOT EXISTS (

 SELECT R.sid FROM Reserves R

 WHERE R.sid = S.sid AND R.bid = B.bid

)

);

21. Consider the following relational schema. Student (id, name, age, city) Retrieve the names of all students.
Display the id's of all students who are having age above 20. Display the names and id's of all students who are having age between 20 and 25 and lives in Hyderabad city.

1. Consider the following relational schema. Student (id, name, age, city)

(i) Retrieve the names of all students.

SELECT name FROM Student;

(ii) Display the id's of all students who are having age above 20.

SELECT id FROM Student WHERE age > 20;

(iii) Display the names and id's of all students who are having age between 20 and 25 and lives in Hyderabad city.

**SELECT id, name FROM Student
WHERE age BETWEEN 20 AND 25
AND city = 'Hyderabad';**

22. Discuss about Complex integrity constraints in SQL.

COMPLEX INTEGRITY CONSTRAINTS IN SQL

A CONSTRAINT can be one of the following:

a. Column-level constraint

Column-level constraints refer to a single column in the table

b. Table-level constraint

Table-level constraints are applied to the whole table.

□ Constraints over a Single Table □

We can specify complex constraints over a single table using table constraints, which have the form **CHECK** conditional-expression. The CHECK constraint is used to limit the value range that can be placed in a column. It performs check on the value before storing them.

For example, to ensure that rating must be an integer in the range 1 to 10, we could use:

```
CREATE TABLE Sailors ( sid INTEGER, sname CHAR(10),
```

```
rating INTEGER, age REAL,
```

```
PRIMARY KEY (sid),
```

```
CHECK (rating >= 1 AND rating <= 10 ))
```

□ Domain Constraints □

A domain is a unique set of values permitted for an attribute in a table. For example, a domain of month-of-year can accept January....December as possible values, a domain of integers can accept whole numbers that are negative, positive and zero.

The data type of domain includes string, character, integer, time, date, currency, etc. The value of the attribute must be available in the corresponding domain.

Example:

ID	NAME	SEMENSTER	AGE
1000	Tom	1 st	17
1001	Johnson	2 nd	24
1002	Leonardo	5 th	21
1003	Kate	3 rd	19
1004	Morgan	8 th	A

Not allowed. Because AGE is an integer attribute

A user can define a new domain using the CREATE DOMAIN statement, which uses CHECK constraints.

Syntax :

```
CREATE DOMAIN domain_name AS data_type
```

```
CHECK (value specification)
```

Example

```
CREATE DOMAIN RATINGVAL AS INT CHECK
```

```
(value >= 0 AND value <= 10);
```

INTEGER is the underlying, or source, type for the domain ratingval, and every ratingval value must be of this type. Values in ratingval are further restricted by using a CHECK constraint. In defining this constraint, we use the keyword VALUE to refer to a value in the domain.

Now RATINGVAL domain is created so, we can use this domain anywhere in any table of database as shown below:

```
CREATE TABLE student( Sid INT PRIMARY KEY,  
name varchar(30), rating RATINGVAL  
);
```

The optional DEFAULT keyword can also be used to associate a default value with a domain. If the domain ratingval is used for a column in some relation and no value is entered for this column, then default value associated with ratingval will be used.

☐ **DEFAULT CONSTRAINT**

The DEFAULT constraint is used to provide a default value for a column. The default value will be added to all new records if no other value is specified.

Example:

```
CREATE TABLE Persons(  
ID int NOT NULL, LastName varchar(25) NOT NULL , FirstName varchar(25), Age int; City  
varchar(25) DEFAULT 'Sandnes'
```

); ☐ Assertions: ICs over Several Tables

Assertions are constraints over several tables. It is a condition that we wish the database should always satisfy.

Syntax

```
CREATE ASSERTION <Constraint name> CHECK (search condition)  
[ <constraint attributes> ]
```

Example:

Suppose the constraint is that the number of boats plus the number of sailors should be less than 100. Here the constraint is on both- boat table as well as on sailor table. In such cases we create assertions.

```
CREATE ASSERTION smallClub  
CHECK (( SELECT COUNT (S.sid) FROM Sailors S )  
+ ( SELECT COUNT (B. bid) FROM Boats B)  
< 100 );
```

23. Define trigger and explain its three parts? Differentiate row level and statement level triggers.

A trigger is a procedure that is automatically invoked by the DBMS in response to specified changes to the database, and is typically specified by the DBA. For example, a trigger can be invoked when a row is inserted into a specified table or when certain table columns are being updated.

A trigger description contains three parts:

- ☐ **Event:** A change to the database that activates the trigger.
- ☐ **Condition:** A query or test that is run when the trigger is activated.
- ☐ **Action:** A procedure that is executed when the trigger is activated and its condition is true.

There are two types of triggers

☐ **Row Level Trigger**

Row Level Trigger is fired each time **row** is affected by Insert, Update or Delete command. To create a new row-level trigger, you use the CREATE TRIGGER statement with the FOR EACH ROW clause.

☐ **Statement-level Trigger**

A statement-level trigger is fired whenever a trigger event occurs on a table regardless of how many rows are affected. In other words, a statement-level trigger executes once for each transaction. By

default, the statement CREATE TRIGGER creates a statement-level trigger when you omit the FOR EACH ROW clause.

Syntax:

```
create trigger [trigger_name] [before | after]
{insert | update | delete} on [table_name]
[for each row]
[trigger_body]
```

Explanation of syntax:

1. create trigger [trigger_name]: Creates or replaces an existing trigger with the trigger_name.
2. [before | after]: This specifies when the trigger will be executed.
3. {insert | update | delete}: This specifies the DML operation.
4. The table_name is the table to which the trigger applies.
5. [for each row]: This specifies a row-level trigger, i.e., the trigger will be executed for each row being affected. In case of statement level trigger this line is simply omitted.
6. [trigger_body]: This provides the operation to be performed as trigger is fired

BEFORE and AFTER of Trigger:

BEFORE triggers run the trigger action before the triggering statement is run. AFTER triggers run the trigger action after the triggering statement is run.

Example program UPDATE TRIGGER

```
CREATE TRIGGER TT BEFORE UPDATE ON STD FOR EACH ROW
BEGIN
IF NEW.marks < 0 THEN
SET NEW.marks = 0;
ELSEIF NEW.marks > 100 THEN SET NEW.marks = 100;
END IF; END;
$$
```

Triggers are mostly used for maintaining the integrity of the information on the database and to avoid redundancy.

Difference Between Row-Level and Statement-Level Triggers

Feature	Row-Level Trigger	Statement-Level Trigger
Execution Frequency	Executes once for each affected row	Executes once per SQL statement, regardless of the number of rows affected
Use Case	Used when actions depend on individual row changes	Used when actions depend on the entire operation rather than specific rows
Access to OLD and NEW values	Has access to <code>OLD</code> and <code>NEW</code> values for each row (in <code>UPDATE</code> and <code>DELETE</code> triggers)	Does not have access to <code>OLD</code> and <code>NEW</code> values since it runs once per statement
Performance	Can be slower for bulk updates/deletes as it fires for each row	More efficient for bulk updates/deletes since it executes only once
Example	Logging changes per row in an audit table	Recording the number of rows modified by an operation

24.Elaborate on Nested Queries and Correlated nested queries in SQL.

NESTED QUERIES

- A nested query is a query that has another query embedded within it. The outer query is known as the main query, and the inner query is known as a subquery.
- The result of inner query is used in execution of outer query.
- A subquery typically appears within the WHERE clause of a query.

There are mainly two types of nested queries:

- **Independent Nested Queries**
- **Co-related Nested Queries**

1. Independent Nested Queries: The execution of inner query is independent of outer query, but the result of inner query is used in execution of outer query. Various operators like IN, NOT IN, ANY, ALL etc are used in writing independent nested queries.

- **IN:** The IN operator is used when you want to compare a column with more than one value. It allows us to test whether a value is in a given set of elements or not.

sid	sname	rating	age
22	Dustin	7	45.0
29	Brutus	1	33.0
31	Lubber	8	55.5
32	Andy	8	25.5
58	Rusty	10	35.0
64	Horatio	7	35.0
71	Zorba	10	16.0
74	Horatio	9	35.0
85	Art	3	25.5
95	Bob	3	63.5

Sailors

sid	bid	day
22	101	10/10/98
22	102	10/10/98
22	103	10/8/98
22	104	10/7/98
31	102	11/10/98
31	103	11/6/98
31	104	11/12/98
64	101	9/5/98
64	102	9/8/98
74	103	9/8/98

Reserves

bid	bname	color
101	Interlake	blue
102	Interlake	red
103	Clipper	green
104	Marine	red

Boats

Example:

(Q1) Find the names of sailors who have reserved boat 103.

```
SELECT S.sname
FROM   Sailors S
WHERE  S.sid IN ( SELECT R.sid
                  FROM   Reserves R
                  WHERE  R.bid = 103 )
```

The nested subquery computes the set of *sids* for sailors who have reserved boat 103 (the set contains 22, 31, and 74), and the top-level query retrieves the names of sailors whose *sid* is in this set.

Result:

Name
Dustin
Lubber
Horatio

- **NOT IN:** The NOT IN operator is used when you want to retrieve a column that has no entries in the table or referencing table.

Example:

To find the names of sailors who have not reserved boat 103, we replace the outermost occurrence of IN by NOT IN

```
SELECT S.sname
FROM   Sailors S
WHERE  S.sid NOT IN ( SELECT R.sid
                     FROM   Reserves R
                     WHERE  R.bid = 103 )
```

Result:

sname
Brutus
Andy
Rusty
Horatio
Zorba
Art
Bob

□ **Multiply Nested Query:** A subquery can contain another query, making it a multiply nested query.

Example:

Find the names of sailors who have reserved a red boat.

```
SELECT S.sname
FROM   Sailors S
WHERE  S.sid IN ( SELECT R.sid
                  FROM   Reserves R
                  WHERE  R.bid IN ( SELECT B.bid
                                  FROM   Boats B
                                  WHERE  B.color = 'red' ) )
```

The innermost subquery finds the set of *bids* of red boats (102 and 104 from Boats). The subquery one level above finds the set of *sids* of sailors who have reserved one of these boats. (22, 31, and 64 from Sailor). The top-level query finds the names of sailors whose *sid* is in this set of *sids*; we get Dustin, Lubber, and Horatio.

2. **Co-related Nested Queries:** In co-related nested queries, the output of inner query depends on the row which is being currently executed in outer query. Operators Exists and Not Exists are used in writing correlated nested queries.

□ **EXISTS:** The EXISTS operator is another set comparison operator, such as IN. It allows us to test whether a set is nonempty (**an implicit comparison with the empty set**)

Example:

Find the names of sailors who have reserved boat number 103.

```
SELECT S.sname
FROM   Sailors S
WHERE  EXISTS ( SELECT *
                FROM   Reserves R
                WHERE  R.bid = 103
                   AND R.sid = S.sid )
```

For each Sailor row *S*, we test whether the set of Reserves rows *R* such that *R.bid* = 103 AND *S.sid* = *R.sid* is nonempty. If so, sailor *S* has reserved boat 103, and we retrieve the name. The subquery clearly depends on the current row *S* and must be re-evaluated for each row in Sailors. The occurrence of *S* in the subquery (in the form of the literal *S.sid*) is called a correlation and such queries are called *correlated queries*.

Result:

Name
Dustin
Lubber
Horatio

(Note that we are omitting S.sid here. That's because we are not testing whether a value (S.sid) is in a given set of elements or not, instead we are testing whether a set is nonempty. The above query also illustrates the use of the special symbol * in situations where all we want to do is to check that a qualifying row exists, and do not really want to retrieve any columns from the row.)

□ **NOT EXISTS** : The EXISTS condition can also be combined with the NOT operator. The NOT operator negates the EXISTS operator.

As a example, by using NOT EXISTS instead of EXISTS, we can compute the names of sailors who have not reserved a red boat.

Example:

```
SELECT S.sname
FROM   Sailors S
WHERE  NOT EXISTS ( SELECT *
                    FROM   Reserves R
                    WHERE  R.bid = 103
                        AND R.sid = S.sid )
```

Result:

sname
Brutus
Andy
Rusty
Horatio
Zorba
Art
Bob